AI-Powered Animated Video Generation System: From Prompt to Polished Short Film with Modular Agents

Umer Farooq[1], Muhammad Usman[2], Muhammad Irtaza Khan[3], and Muhammad Ahmed Mufti[4]

National University of Computer and Emerging Sciences (FAST NUCES), Islamabad
i220891@nu.edu.pk

Abstract. We present an end-to-end system that converts a single natural-language prompt into a short animated video with dialogue, character casting, synthesized audio, generated footage, optional lip-sync, and final composition. The system is designed as a modular, phase-oriented pipeline with strict typed contracts enforced using Pydantic. A FastAPI + React web application orchestrates execution, reports real-time status, and enables post-generation edits with versioned undo/redo via file-based state snapshots. We describe the architecture, phase-wise implementation, shared schema, state versioning mechanism, and evaluation-oriented deliverables.

Keywords: Agentic AI · LLM orchestration · Text-to-video · TTS · State versioning · Undo/redo

1 Introduction

Generating even short animated videos traditionally requires multiple specialist roles and significant production time. Recent progress in generative language, audio, and vision models enables automation, but building a student-accessible pipeline requires careful orchestration, modularity, and robust contracts. This project implements a full-stack system that: (i) expands a raw user prompt into a structured screenplay, (ii) synthesizes character voices and a timing manifest, (iii) generates per-scene video clips and optionally performs lip-sync, and (iv) composites a final MP4. Finally, an editing interface supports natural-language edit requests and full undo/redo through versioned snapshots.

2 System Overview

2.1 Key Requirements (Project Brief Alignment)

The system was designed to satisfy the course project phases:

– **Phase 1: Story & Script** — prompt → structured scenes, dialogue, and character metadata.

- **Phase 2: Audio** — TTS synthesis and a per-scene timing manifest.
- **Phase 3: Video** — per-scene generation, A/V synchronization, and export.
- **Phase 4: Web Interface** — orchestration, progress visibility, preview and download.
- **Phase 5: Edit + Undo** — interpret edit commands, apply targeted regeneration/recomposition, and maintain version history.

2.2 Architecture Diagram

Figure 1 summarizes the pipeline modules and the shared state contract passed across phases.

3 Shared Data Contracts

All modules communicate via a single typed state model, minimizing integration ambiguity and enabling snapshotting.

3.1 Core Models

We use Pydantic v2 models for strict validation.

Story output. Scenes contain characters, dialogue, and visual cues for downstream video prompting. Character metadata includes gender and free-text descriptions to support voice and portrait consistency.

Audio timing. For each scene we store the audio file path, duration, subtitles (optional), and a dialogue timeline with start/end timestamps in seconds. This timeline can drive segment-wise lip-sync.

Project state. A single `ProjectState` contains the latest story object, timing manifest, version counter, and an asset map of generated files.

4 Phase-wise Implementation

4.1 Phase 1: Story, Script & Character Design

The story module expands a prompt into a structured screenplay. It implements a reliability-first provider cascade:

- **Groq (cloud)** when enabled (fast inference).
- **Ollama (local)** when available for offline runs.
- **Deterministic local fallback** to ensure the pipeline never blocks.

The final output is converted into `StoryOutput` (Pydantic) to prevent unstructured downstream coupling.

4.2 Phase 1.5: Casting Portraits

Character portraits are generated using a lightweight web image generation API (Pollinations). The orchestrator extracts unique characters across scenes and generates a portrait per character for UI display and identity anchoring.

4.3 Phase 2: Audio Generation

Audio is synthesized using Edge-TTS to produce neural speech without requiring paid keys. For each scene, we generate a final mixed audio file plus a dialogue timeline. The pipeline stores `audio_paths` in `state.assets` and a structured `TimingManifest` in `state.timing`.

4.4 Phase 3: Video Generation

For each scene, the video agent builds a rich prompt from: (i) location, (ii) character descriptions, and (iii) dialogue visual cues. Video clips are generated via Wan2.1 served through a Gradio UI, invoked programmatically by a client. If the generator is unreachable, the pipeline falls back to placeholder clips to preserve end-to-end demonstrability.

4.5 Phase 3.5: Lip-Sync

The lip-sync agent implements a robust strategy:

- If a dialogue timeline exists: split the video into segments by timestamps, apply Wav2Lip (if installed) or FFmpeg mux per segment, and merge.
- If no timeline exists: apply whole-scene Wav2Lip or FFmpeg mux fallback.

This improves speaker alignment and reduces common failure modes where the first detected face animates across all dialogue.

4.6 Phase 4: Composition

The compositor stitches scene clips, aligns audio, and exports `final_output.mp4`. Outputs are served via the backend as static files for in-browser preview and download.

5 Web Orchestration (Phase 4)

5.1 Backend (FastAPI)

The backend provides:

- `POST /api/generate`: start the full pipeline as a background task.
- `GET /api/status/{job}`: poll job status/progress and provide portrait paths.
- `POST /api/edit`: apply an edit request to the latest state snapshot.
- `POST /api/undo`, `POST /api/redo`: state version navigation.

Progress is persisted in `data/temp/<job>.json` to support simple polling and resilience.

5.2 Frontend (React + Vite)

The UI provides prompt entry, a phase-aware progress display, cast gallery, a video monitor, and a free-text edit console. Undo/redo buttons call backend endpoints and refresh the video with cache-busting query parameters.

6 Edit Agent and Versioned Undo (Phase 5)

6.1 Intent Parsing and Scoped Execution

Edit queries are converted into a structured intent object (target, scope, parameters). For the most common use case (scene-specific visual changes), the executor:

1. creates an undo snapshot of the current state and assets,
2. updates the targeted scene prompt,
3. regenerates only the affected scene clip,
4. recomposites the final video.

6.2 State Snapshot Store

Every run or edit creates a versioned snapshot:

- `data/state_versions/vX/state.json` — serialized `ProjectState`.
- `data/state_versions/vX/assets/` — full copy of `data/outputs/`.

Undo/redo restores both the structured state and the physical assets, ensuring reliable replay of previous outputs.

7 Sequence Diagram (End-to-End Run)

Figure 2 shows the main runtime sequence from a user prompt through the pipeline and snapshot.

8 Testing Strategy

The repository includes phase-level scripts for story, audio, pipeline integration, and edit/undo flows. For grading-oriented completeness, an ideal next step is to convert these into a structured unit test suite (e.g., pytest) and include an intent-classification matrix for at least ten edit query types.

Table 1: Primary technologies used in the implementation.

Layer	Tech	Purpose
Backend	FastAPI + Uvicorn	orchestration endpoints, background tasks
Frontend	React + Vite	prompt UI, progress, preview, edit/undo
Schemas	Pydantic v2	strict typed contracts between phases
LLM	Groq / Ollama / Local	screenplay generation with fallbacks
TTS	Edge-TTS	free neural voice synthesis
Portraits	Pollinations API	character casting images
Video Gen	Wan2.1 via Gradio client	per-scene footage generation
Lip-sync	Wav2Lip + FFmpeg fallback	speaker/scene alignment
Composition	MoviePy + FFmpeg	stitch clips and mux audio
State Store	file-based snapshots	version history + undo/redo

9 Technology Stack

10 Configuration and Reproducibility

The system is configured through environment variables loaded from a local `.env` file. For security, secrets (e.g., API keys) should not be committed; instead, a `.env.example` can document the expected variables. The configuration used in this implementation is summarized below:

- **LLM providers (Phase 1)**: Groq is optionally supported, but the current configuration uses Ollama locally (`GROQ_ENABLED=0`, `OLLAMA_ENABLED=1`) with `OLLAMA_MODEL=llama3.1:8b` and `OLLAMA_BASE_URL=http://localhost:11434`.
- **Audio (Phase 2)**: Edge-TTS is selected (`STUDIO_FLOOR_VOICE_BACKEND=edge_tts`), with rate/pitch controls (`STUDIO_FLOOR_EDGE_TTS_RATE`, `STUDIO_FLOOR_EDGE_TTS_PITCH`).
- **Video generation (Phase 3)**: the Wan2GP Gradio server is addressed via `GRADIO_URL=http://127.0.0.1:42003/`. Some Gradio deployments expose a save-inputs endpoint; this setup uses `GRADIO_SAVE_INPUTS_API_NAME=/save_inputs_14`.

11 Results and Evidence

The system produces:

- A final composited MP4 at `data/outputs/final_output.mp4`.
- Versioned snapshots at `data/state_versions/v1...vN/`, each containing `state.json` and full asset copies.

These artifacts support demonstration of multi-edit workflows and reliable undo/revert operations.

12 Limitations and Future Work

- **Intent parsing**: extending from rule-based parsing to an LLM-based structured classifier (e.g., LangGraph) would better cover diverse edit intents (audio, script, compositing options).
- **Phase-level reruns**: adding explicit endpoints/UI buttons for audio-only regeneration, video-only regeneration, and recompose-only would improve usability and alignment with the brief.
- **Testing**: converting scripts into a CI-ready unit test suite and adding a 10+ intent test matrix would strengthen the Phase 5 deliverable.

A Schema Appendix

A.1 StoryOutput (Representative JSON)

```
{
  "scenes": [
    {
      "scene_id": 1,
      "location": "Rainy Tokyo alleyway",
      "characters": ["Narrator", "Agent K"],
      "character_metadata": {
        "Narrator": { "name": "Narrator", "gender": "male", "description":
             "calm voice" },
        "Agent K": { "name": "Agent K", "gender": "female", "description":
             "sharp, observant spy" }
      },
      "dialogue": [
        { "speaker": "Narrator", "line": "The night hides secrets.", "
            visual_cue": "neon reflections" }
      ]
    }
  ],
  "character_metadata": {
    "Narrator": { "name": "Narrator", "gender": "male", "description": "
        calm voice" }
  }
}
```

A.2 TimingManifest (Representative JSON)

```
{
  "scene_timings": {
    "1": {
      "audio_filepath": "data/outputs/audio/scene_01.wav",
      "duration_ms": 8200,
```

```
      "subtitle_text": "The night hides secrets.",
      "speaker_tracks": {},
      "dialogue_timeline": [
        { "speaker": "Narrator", "line": "The night hides secrets.", "
            start_seconds": 0.0, "end_seconds": 3.2 }
      ]
    }
  }
}
```

A.3 ProjectState (Representative JSON)

```
{
  "version": 5,
  "story": { "...": "StoryOutput" },
  "timing": { "...": "TimingManifest" },
  "assets": {
    "audio_paths": { "1": "data/outputs/audio/scene_01.wav" },
    "video_paths": { "1": "data/outputs/scenes/scene_1.mp4" },
    "synced_video_paths": { "1": "data/outputs/synced/scene_1_synced.mp4"
        },
    "character_portraits": { "Agent K": "outputs/characters/agent_k.png"
        }
  }
}
```

B Environment Variables (Appendix)

```
# LLM Providers for Story Agent (Phase 1)
GROQ_ENABLED=0
GROQ_API_KEY=
GROQ_MODEL=llama-3.1-8b-instant
GROQ_BASE_URL=https://api.groq.com/openai/v1

OLLAMA_ENABLED=1
OLLAMA_MODEL=llama3.1:8b
OLLAMA_BASE_URL=http://localhost:11434

# Audio (Phase 2)
STUDIO_FLOOR_VOICE_BACKEND=edge_tts
STUDIO_FLOOR_EDGE_TTS_RATE=+0%
STUDIO_FLOOR_EDGE_TTS_PITCH=+0Hz

# Video Generation (Phase 3 - Wan2GP via Pinokio)
GRADIO_URL=http://127.0.0.1:42003/
GRADIO_SAVE_INPUTS_API_NAME=/save_inputs_14
```

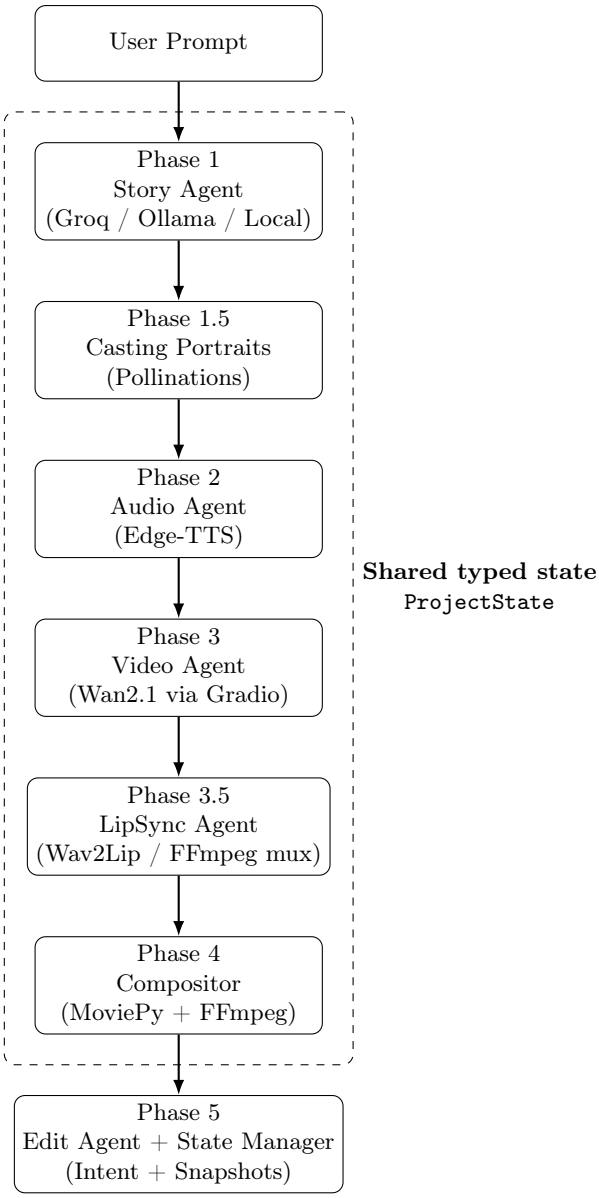

Fig. 1: High-level architecture. Each phase consumes/produces a typed `ProjectState` and assets under `data/outputs/`.

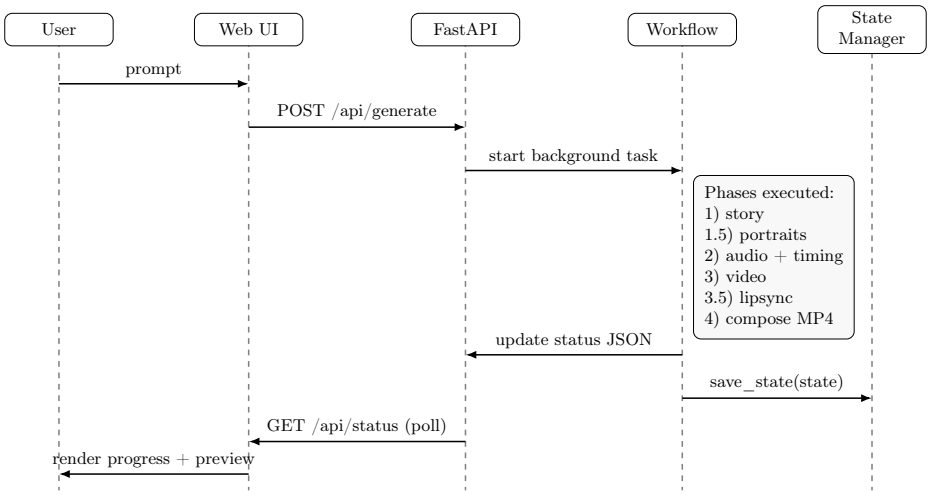

Fig. 2: End-to-end runtime sequence: prompt submission, background orchestration, progress polling, and state snapshotting.